



Security Audit

Report for RWAToken

Contracts

Date: October 9, 2026 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction	1
1.1 About the Audit Target	1
1.2 Disclaimer	1
1.3 Procedure of Auditing	2
1.3.1 Security Issues	2
1.3.2 Additional Recommendation	3
1.4 Security Model	3
Chapter 2 Findings	4
2.1 Note	4
2.1.1 Ensure enough funds to pay users in <code>redeemVault</code>	4
2.1.2 Off-chain recalculation of zero <code>uiAmount</code>	4
2.1.3 Dust visibility due to precision loss	4
2.1.4 Off-chain operational alignment after the upgrade	5
2.1.5 Potential centralization risks	5
2.1.6 Price deviation between attestation issuance and execution	6
2.1.7 Assumption on attestation price calculation	6
2.1.8 Minting operation after delisting	6

Report Manifest

Item	Description
Client	Bitget
Target	RWAToken Contracts

Version History

Version	Date	Description
1.0	October 9, 2026	First release

Signature



Digitally signed by: BlockSec (<https://blocksec.com>)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2026-10-09 11:02:51+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at [Email](#), [Twitter](#) and [Medium](#).

Chapter 1 Introduction

1.1 About the Audit Target

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The audit target (hereinafter referred to as the Target) of this audit is the code repository ¹ of RWAToken Contracts of Bitget.

The rwa-evm project replaces rebase accounting with an ERC-8056 Scaled UI amount mechanism. The mutable rebase index is dropped in favor of a UI multiplier: the ledger becomes a fixed 1:1 share book at constant scale, tokens initialize with a multiplier rather than a starting index, and corporate-action splits adjust the displayed amount rather than the underlying shares. Separately, a new upgradeable contract, RWATokenTrade, lets users mint and redeem RWA tokens 24/7, without market-hour restrictions.

Note this audit only focuses on the codes located in the following directories/files.

- src/RWAToken.sol
- src/RWATokenRebaseUpgradeable.sol
- src/RWATokenScaledUIUpgradeable.sol
- src/RWATokenTrade.sol
- src/interfaces/

Other files are not within the scope of the audit. Additionally, all dependencies of the Target are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version ([Version 1](#)), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version ([Version 0](#)), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
rwa-evm	Version 0	d607918fe884eba4fa7a54d194acb23f071c58ba
	Version 1	8ac0f24cbc0c62432526dc4161d0ce2b83aab3cd

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on,

¹<https://github.com/blcokchainbg/rwa-evm>

the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the Target, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of the Target.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan the Target with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of the Target and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

- * Access control
- * Permission management
- * Whitelist and blacklist mechanisms
- * Initialization consistency
- * Improper use of the proxy system
- * Reentrancy
- * Denial of Service (DoS)
- * Untrusted external call and control flow
- * Exception handling
- * Data handling and flow
- * Events operation
- * Error-prone randomness
- * Oracle security
- * Business logic correctness
- * Semantic and functional consistency

- * Emergency mechanism
- * Economic and incentive impact

1.3.2 Additional Recommendation

- * Gas optimization
- * Code quality and style



Note The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology² and Common Weakness Enumeration³. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	High	High	Medium
	Low	Medium	Low
		High	Low
		Likelihood	

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

²https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

³<https://cwe.mitre.org/>

Chapter 2 Findings

In total, we have **eight** notes.

- Note: 8

ID	Severity	Description	Category	Status
1	-	Ensure enough funds to pay users in <code>redeemVault</code>	Note	-
2	-	Off-chain recalculation of zero <code>uiAmount</code>	Note	-
3	-	Dust visibility due to precision loss	Note	-
4	-	Off-chain operational alignment after the upgrade	Note	-
5	-	Potential centralization risks	Note	-
6	-	Price deviation between attestation issuance and execution	Note	-
7	-	Assumption on attestation price calculation	Note	-
8	-	Minting operation after delisting	Note	-

The details are provided in the following sections.

2.1 Note

2.1.1 Ensure enough funds to pay users in `redeemVault`

Introduced by `Version 1`

Description In contract `RWATokenTrade`, function `redeemWithSignature()` relies on the address `redeemVault` to provide the necessary settlement tokens for user redemptions. Therefore, the project should ensure that the address `redeemVault` has sufficient token balances and allowances.

2.1.2 Off-chain recalculation of zero `uiAmount`

Introduced by `Version 1`

Description In contract `RWAToken`, function `_postTokenTransfer()` emits event `TransferWithUIAmount` after each transfer, mint, or burn operation. The `uiAmount` is calculated via function `_tryToUIAmount()`, which falls back to zero when `amount * uiMultiplier` overflows. In this case, the transaction still succeeds, but the event records `uiAmount` as zero. Therefore, when the emitted `uiAmount` is zero while `amount` is non-zero, off-chain services (e.g., the frontend) should recalculate the UI amount.

Feedback from the project The project stated that the off-chain service can properly handle this fallback scenario.

2.1.3 Dust visibility due to precision loss

Introduced by `Version 1`

Description In contract `ERC8056BaseUpgradeable`, function `balanceOfUI()` returns the UI-adjusted balance as `rawAmount * uiMultiplier() / MULTIPLIER_DECIMALS`, where the result is rounded down. When the function `uiMultiplier()` returns a value smaller than `MULTIPLIER_DECIMALS`, a small raw balance (i.e., dust) rounds down. This dust becomes invisible in the UI value, even though the raw balance remains fully preserved in storage. Integrators should rely on the raw balance for accounting and treat the UI amount only as a display value.

Feedback from the project The project clarifies that function `balanceOfUI()` is only used to display, while accounting relies on function `balanceOf()`.

2.1.4 Off-chain operational alignment after the upgrade

Introduced by [Version 1](#)

Description The upgrade moves the token ledger from the rebase model to the ScaledUI model. After the upgrade, contracts no longer enforce consistency between the two representations, so the following points must be handled off-chain to keep on-chain and off-chain records aligned.

1. The `uiMultiplier` must be aligned with the legacy `rebaseIndex` for each upgradable token. The initialization of `RWATokenScaledUIUpgradeable` only checks the bounds and does not verify alignment with the pre-upgrade `rebaseIndex`, so the value must be derived from the `rebaseIndex` such that function `balanceOfUI()` reproduces the pre-upgrade displayed balances exactly.

2. For off-chain operators, reconciliation must move from the raw representation to the UI representation. Off-chain ledgers and event consumers should read from the updated interfaces such as functions `balanceOfUI()`, `totalSupplyUI()`, and `toUIAmount()`.

3. Contract `RWATokenManager` has fully operational functions `mintWithSignature()` and `redeemWithSignature()`, while contract `RWATokenTrade` also implements functions with the same names. These contracts enforce different logic. Therefore, the project should deprecate these functions in contract `RWATokenManager` after contract `RWATokenTrade` is deployed.

If any of these steps are missed, off-chain records diverge from the on-chain display representation, causing reconciliation breaks, incorrect deposit or withdrawal instructions, or user-facing balance disputes.

Feedback from the project The project states that they will set each token's post-upgrade `uiMultiplier` based on its existing `rebaseIndex`, ensuring `balanceOfUI()` remains continuous. Additionally, the project will revoke the corresponding signer authorization to deprecate functions `mintWithSignature()` and `redeemWithSignature()` of contract `RWATokenManager` after `RWATokenTrade` is deployed.

2.1.5 Potential centralization risks

Introduced by [Version 1](#)

Description In this project, several privileged roles (e.g., `DEFAULT_ADMIN_ROLE` and `CONFIGURE_ROLE`) can conduct sensitive operations, which introduces potential centralization risks. For example, `DEFAULT_ADMIN_ROLE` controls role assignment and custody configuration, while `CONF`

`GURER_ROLE` executes corporate - action splits through the action oracle, delists tokens and freezes individual accounts. If privileged accounts' private keys are lost or maliciously exploited, this could pose a significant risk to the protocol.

Feedback from the project The project states that privileged roles are managed through multisig wallets, with least-privilege access controls and separation of duties. Private keys are stored offline, and key rotation procedures are available in emergencies.

2.1.6 Price deviation between attestation issuance and execution

Introduced by [Version 1](#)

Description Users obtain attestations off-chain and then submit them on-chain to execute trades. Since these two steps are asynchronous, the signed price may deviate from the real-time price at execution. Contract `SanityCheckOracle` limits this deviation via variable `maxDeviationBps`. Therefore, the project should configure this variable appropriately. Notably, even if the deviation is within the allowed range, the project may still incur losses due to the price difference. For example, assume `maxDeviationBps` is set to 500 (i.e., 5%) and the signed price is \$100, leaving the project exposed to the \$5 price difference per share. If the real-time price rises to \$105, users can still mint tokens at \$100. Therefore, the project should implement off-chain monitoring and reserve an adequate price buffer when issuing attestations.

Feedback from the project The project states that it will configure `maxDeviationBps` and related parameters properly, use off-chain real-time monitoring, and maintain an adequate quote buffer. Additionally, they will shorten attestation validity when needed.

2.1.7 Assumption on attestation price calculation

Introduced by [Version 1](#)

Description In contract `RWATokenTrade`, functions `mintWithSignature()` and `redeemWithSignature()` calculate the USD value of a trade as `quantity * price`, where `quantity` is the amount of `RWAToken`. According to ERC-8056, it represents `quantity * uiMultiplier / 1e18` shares. Therefore, the `price` should be the per-share price adjusted by `uiMultiplier / 1e18`. However, function `SanityCheckOracle._validateViaPyth()` checks the `price` against the real-time Pyth Lazer quote, which is a per-share price and is incompatible with the current design. As a result, the project should disable the Pyth path and ensure that the price is converted from a per-share price using the on-chain `uiMultiplier`. Additionally, the `uiMultiplier` should be updated promptly to reflect stock splits and reverse splits of the underlying RWA.

Feedback from the project The project states that the Pyth validation path will be disabled through configuration and that the self-hosted price oracle will convert per-share prices using the on-chain `uiMultiplier`. Additionally, updates to `uiMultiplier` are triggered by actual corporate actions.

2.1.8 Minting operation after delisting

Introduced by [Version 1](#)

Description In contract [RWAToken](#), function `mint()` does not check whether the token has been delisted. Consequently, the minting operation can be executed even if the token has been delisted, which the business does not expect. The project should stop issuing mint attestations and other mint-related authorizations once a token has been delisted.

Feedback from the project The project states that delisting is only an off-chain status flag and that minting operations will be stopped by off-chain systems.

